IPA ASSIGNMENT – 26

Name – Nishant Nahar

Roll No – 241551078

1. Canny Edge Detection

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("img_2.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:   raise FileNotFoundError("img_1.jpeg not found")
edges = cv2.Canny(img, 100, 200)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

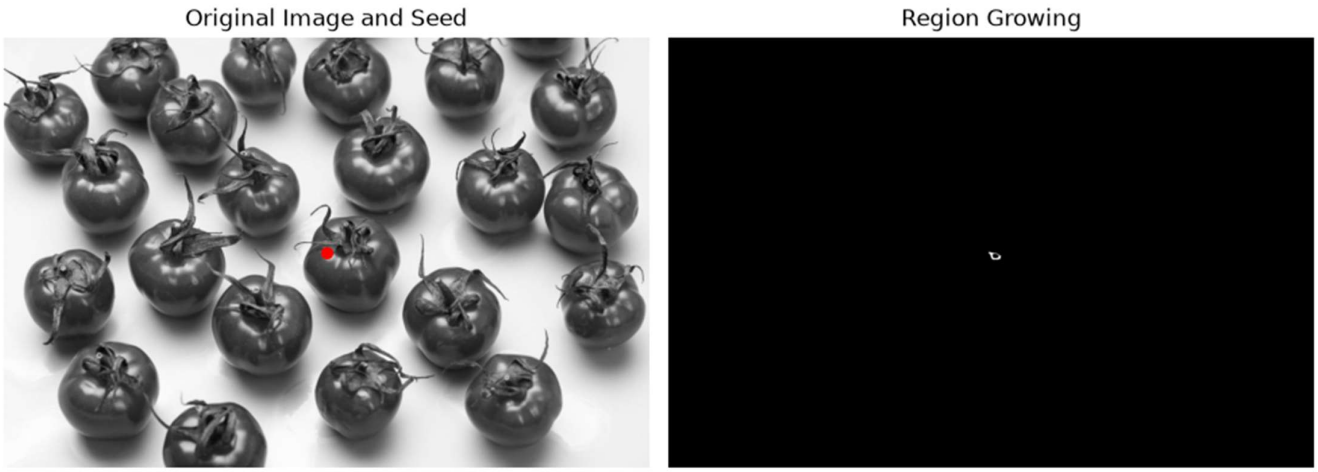


2. Region Growing

```
import cv2 import numpy as np
import matplotlib.pyplot as plt
from collections import deque
img = cv2.imread("img_2.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:  raise FileNotFoundError("img_1.jpeg not found")
def region_growing(img, seed, threshold=15):
    h, w = img.shape
    visited = np.zeros((h, w), dtype=bool)
    region = np.zeros((h, w), dtype=np.uint8)
    sy, sx = seed
    seed_value = int(img[sy, sx])
    queue = deque([(sy, sx)])
    visited[sy, sx] = True
```

```
    neighbors = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    while queue:
        y, x = queue.popleft()
        region[y, x] = 255
        for dy, dx in neighbors:
            ny, nx = y + dy, x + dx
            if 0 <= ny < h and 0 <= nx < w and not visited[ny, nx]:
                visited[ny, nx] = True
                if abs(int(img[ny, nx]) - seed_value) <= threshold:
                    queue.append((ny, nx))
    return region
seed = (img.shape[0] // 2, img.shape[1] // 2)
result = region_growing(img, seed, threshold=15)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.scatter(seed[1], seed[0], color="red", s=30)
plt.title("Original Image and Seed")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(result, cmap="gray")
plt.title("Region Growing")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



3. Image Splitting

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("img_2.jpeg")
if img is None: raise FileNotFoundError("img_1.jpeg not found")
h, w = img.shape[:2]
mid_y, mid_x = h // 2, w // 2
top_left = img[:mid_y, :mid_x]
top_right = img[:mid_y, mid_x:]
bottom_left = img[mid_y:, :mid_x]
bottom_right = img[mid_y:, mid_x:]
titles = ["Top Left", "Top Right", "Bottom Left", "Bottom Right"]
```

```
images = [top_left, top_right, bottom_left, bottom_right]
plt.figure(figsize=(10, 8))
for i, (part, title) in enumerate(zip(images, titles), 1):
    plt.subplot(2, 2, i)
    plt.imshow(cv2.cvtColor(part, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



4. Image Merging

```
import cv2 import matplotlib.pyplot as plt
import numpy as np
img = cv2.imread("img_2.jpeg")
if img is None: raise FileNotFoundError("img_1.jpeg not found")
h, w = img.shape[:2]
mid_y, mid_x = h // 2, w // 2
top_left = img[:mid_y, :mid_x]
top_right = img[:mid_y, mid_x:]
bottom_left = img[mid_y:, :mid_x]
bottom_right = img[mid_y:, mid_x:]
top = np.hstack((top_left, top_right))
```

```
bottom = np.hstack((bottom_left, bottom_right))
merged = np.vstack((top, bottom))
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(merged, cv2.COLOR_BGR2RGB))
plt.title("Merged Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

